

程设大作业 HWpilot 项目作业报告

组号：118 组 成员：宋睿宇，杨若辰

2026 年 7 月 5 日

验收前重要说明：DeepSeek API Key 配置

本项目的 AI 分析、反馈复查和启发式问题生成功能均依赖 DeepSeek API Key。为避免密钥泄露，真实 API Key 不应提交到 GitHub 仓库；助教验收时需要在本地运行程序后手动填写。

为方便验收，仓库根目录提供了一个样例作业文件夹 `test/`。助教启动程序后可以直接选择 `test/` 文件夹，进行项目各功能的验收和测试。

推荐配置步骤

1. 准备一个可用的 DeepSeek API Key。平台网址为 <https://platform.deepseek.com>。
2. 按 README 中的命令编译并启动 HWpilot。
3. 点击左上角“打开文件夹”，选择一个需要分析的作业项目目录；验收时可直接选择随项目提供的 `test/` 文件夹。
4. 程序首次打开该作业目录时，如果没有保存 API Key，会弹出“填写 API Key”窗口。
5. 将 DeepSeek API Key 粘贴到输入框，点击“确认”。
6. 程序会把密钥保存到被分析作业目录下的 `.hwpilot/project.json` 中，之后同一目录可直接使用 AI 功能。
7. 如果首次弹窗中选择了“暂不填写”，可在菜单栏选择“设置 → 设置 API-key”重新填写。

目录

1 程序功能介绍	3
1.1 作业项目打开与数据管理	3
1.2 文件扫描	3
1.3 AI 反馈生成	3
1.4 反馈保存、状态跟踪与复查	3
1.5 启发式问题生成	4
1.6 版本快照与变更查看	4
1.7 菜单栏和辅助功能	4
2 项目各模块与类设计细节	4
2.1 整体架构	5
2.2 ProjectData	5
2.3 ProjectManager	5
2.4 HWFileScanner 文件扫描	5
2.5 HWpilotLLM 大模型交互	6
2.6 MainWindowPrivate 连接 ui 界面与后端逻辑	6
2.7 FeedbackStore 状态记忆	6
2.8 GitService 版本控制	6
2.9 AppText、Render 辅助	6
3 小组成员分工情况	7
4 项目总结与反思	7
4.1 设计收获	7
4.2 不足与改进方向	7

1 程序功能介绍

HWpilot 是一个面向程序设计实习、计算概论等编程基础课程作业场景的辅助工具。项目使用 C++ 与 Qt 开发，界面由 Qt Widgets 完成，网络请求由 Qt Network 完成。程序的核心目标是：在学生完成一门课程作业的整个过程中，帮助记录与跟踪学生与大模型的交互进程，从而帮助学生更方便地复习错题与攻克难题；同时项目也调用了真实的 Git 命令，希望用户初步建立版本控制的思想。

1.1 作业项目打开与数据管理

用户点击“打开文件夹”可以选择一个作业目录，程序会把该目录看作被分析的作业文件夹，并在该目录下创建 `.hwpilot` 数据文件夹，其中，`.hwpilot/project.json` 用于保存项目名、项目路径、最近打开时间、作业要求文本以及 DeepSeek API Key。

如果被打开的作业目录不是 Git 仓库，程序会自动尝试执行 `git init`。这一步力求用户建立版本控制的思想。

1.2 文件扫描

程序会递归地扫描被打开目录中的代码与文本文件，支持 C/C++、Python、Java、JavaScript/TypeScript、Go、Rust、C#、Shell、CMake、Markdown 和 TXT 等常见文件类型。扫描时会跳过隐藏目录和隐藏文件，例如 `.git` 与 `.hwpilot`，并默认跳过超过 200KB 的单个文件，避免一次发送给大模型的上下文过大。

扫描结果会按相对路径排序并展示为文件树。用户可以勾选需要参与 AI 分析的文件，也可以一键全选。

1.3 AI 反馈生成

在“AI 反馈”页中，用户可以填写作业要求和补充问题，并选择要分析的代码文件。程序会把作业要求、用户问题和勾选文件内容组织成 prompt，调用 DeepSeek 大模型 `https://api.deepseek.com/chat/completions`，模型为 `deepseek-chat`。

AI 返回的内容严格采用 JSON 格式，包含简短摘要、问题列表和完整的自然语言报告。程序会优先解析结构化结果，若解析失败则保存原始文本，保证用户仍可查看回复。

1.4 反馈保存、状态跟踪与复查

用户可以把一次 AI 回复保存为反馈记录。记录会写入被分析作业目录的 `.hwpilot/feedbacks.json`。每条反馈包含创建时间、关联的版本、反馈模式、摘要、原始内容和问题列表。

对于问题列表，程序会保存严重程度、文件路径、行号、类别、标题、建议和处理状态。状态包括“未解决”“已解决”“无法确定”和“忽略”。用户可在界面中直接修改状态，修改会同步写回本地 JSON 文件。另外，为了美观，我们将各个状态进行了染色。

程序还支持“反馈复查”。用户可以勾选历史反馈中的待处理条目，再让 AI 结合当前代码判断这些问题是否已经解决。复查记录会与原反馈条目建立关联，并可选择把原条目标记为已解决。

1.5 启发式问题生成

除了直接指出问题，HWpilot 还有“启发式问题”功能。该模式会根据作业要求和选中文件生成开放式问题，帮助学生从题目理解、边界条件、数据结构、算法设计等角度思考。该模式的 prompt 明确要求模型避免直接给完整答案，适合教学辅助场景。

1.6 版本快照与变更查看

左侧版本树展示当前工作区和 Git 历史提交。用户可以点击“提交”保存当前工作目录的快照，程序会收集除 .hwpilot 外的变更，执行 `git add` 和 `git commit -m 「commit message」`。

点击所打开的文件夹名称，会出现版本概览页面，可以展示当前工作区的变更统计。反馈记录会根据当前工作区或具体 commit 进行分组，方便用户追踪“哪个版本产生了哪些反馈”。

1.7 菜单栏和辅助功能

菜单栏提供最近打开项目、语言切换、DeepSeek API Key 设置和大模型温度设置。语言文本由统一的 AppText 表维护，支持中文和英文显示。

2 项目各模块与类设计细节

从宏观上来看，ui 界面逻辑写在 `MainWindow` 与 `MainWindowPrivate` 两个类中，其余功能被分别封装为独立的类。

2.1 整体架构

模块	职责
<code>main.cpp</code>	创建 <code>QApplication</code> , 显示主窗口。
<code>MainWindow</code>	对外的主窗口类, 有 <code>MainWindowPrivate</code> 。
<code>MainWindowPrivate</code>	负责界面构建、按钮响应、文件选择、AI 调用、反馈保存、版本树刷新等。
<code>HWFileScanner</code>	扫描作业目录, 读取支持类型的文件, 格式化发给 LLM 的代码内容。
<code>HPilotLLM</code>	处理与 DeepSeek 的聊天 prompt 补全, 并处理大模型的回复。
<code>ProjectManager</code>	维护 <code>.hwpilot/project.json</code> 项目数据。
<code>FeedbackStore</code>	维护 <code>.hwpilot/feedbacks.json</code> , 负责反馈增删改查中的保存、导入和状态更新。
<code>GitService</code>	以 <code>QProcess</code> 调用真实 Git, 封装 <code>status</code> 、 <code>diff</code> 、 <code>log</code> 、 <code>add</code> 、 <code>commit</code> 等操作。
<code>ProjectData</code>	定义 <code>ProjectData</code> 、 <code>FeedbackRecord</code> 、 <code>FeedbackItem</code> 三个数据结构及 JSON 序列化。
<code>AppText</code>	统一管理中英文界面文字和 prompt 内容。
<code>Render</code>	将所有内容渲染为界面可显示的 HTML。

2.2 ProjectData

`ProjectData` 反映项目状态, 包括项目名称、项目路径、作业要求、API Key、最近打开时间和反馈记录列表。`FeedbackRecord` 表示一次 AI 回复, 保存记录 ID、关联 commit、模式、创建时间、摘要、原始内容和解析状态。`FeedbackItem` 表示反馈中的单条问题, 包含严重程度、文件路径、行号、类别、标题、建议、处理状态和复查关联。

这个设计使得 AI 返回的自然语言报告和问题列表能够同时保存。

2.3 ProjectManager

`ProjectManager::openProject` 会检查作业目录是否存在, 创建 `.hwpilot` 目录, 读取已有 `project.json`, 并更新项目路径与最近打开时间。保存时使用 `QJsonDocument::Indented` 输出, 便于用户手动检查。

API Key 存储在 `ProjectData::apiKey` 中。主界面打开项目后, 如果其为空, 就调用 API Key 输入对话框。也就是 API Key 被存储在被分析的作业文件夹中, 而不是存在项目运行时占用的内存中。

2.4 HWFileScanner 文件扫描

`HWFileScanner::scanDirectory` 使用 `QDirIterator` 递归遍历文件。它只读取适合发给 LLM 的文件。扫描过程中会跳过隐藏路径, 并限制单文件大小。每个文件被转换为 `CodeFile`, 保存绝

对路径、相对路径、内容和扩展名。

`formatFilesForLLM` 会把作业要求和每个文件内容拼成 Markdown 代码块。相对路径会写在每个代码块前，便于大模型在反馈中定位文件。

2.5 HWpilotLLM 大模型交互

HWpilotLLM 在构造函数中接收 API Key，并设置 DeepSeek 接口地址。发送请求时，它构造 JSON 请求，设置 `model` 为 `deepseek-chat`，并通过 `Authorization: Bearer <key>` 请求头来表明身份。

网络请求使用 Qt 的异步 `QNetworkAccessManager`，解析 OpenAI 兼容格式中的 `choices[0].message.content`。如果 Qt 网络请求在无 HTTP 状态码的情况下失败，会尝试使用 `curl` 作为 fallback，防止与大模型交互意外崩掉。

2.6 MainWindowPrivate 连接 ui 界面与后端逻辑

MainWindowPrivate 是项目中负责协调的类。它构建左侧版本树、AI 分析页、启发式问题页和反馈记录页，并把 UI 事件连接到各模块。

打开项目时，流程为：选择目录、调用 `ProjectManager` 初始化数据、调用 `FeedbackStore` 打开反馈仓库、必要时初始化 Git、提示填写 API Key、扫描文件、刷新版本树。

执行 AI 分析时，流程为：重新扫描项目、读取勾选文件、检查 API Key、保存作业要求、整理用户问题和选中反馈历史、构造 `system/user messages`、调用 HWpilotLLM、接收回复并显示。保存反馈时，程序调用 `buildFeedbackRecord` 解析 JSON，把结果写入 `FeedbackStore`。

2.7 FeedbackStore 状态记忆

FeedbackStore 专门维护 `feedbacks.json`。它支持添加反馈、更新单条问题状态、批量更新状态、重命名反馈记录、标记问题已复查，以及在 Git 提交后把原本关联当前工作区的反馈重关联到新 commit。

这种设计使反馈记录不依赖程序运行时内存，关闭程序后仍能在下次打开同一作业目录时恢复。

2.8 GitService 版本控制

GitService 通过 `QProcess` 调用命令行 Git。

项目在 Git `paths` 中排除了 `.hwpilot/**`，避免把本地反馈记录和 API Key 误提交到作业版本快照中。提交时只添加真实作业文件的变化，再根据新 HEAD 把当前工作区反馈重新关联到新的提交中。

2.9 AppText、Render 辅助

AppText 用表格的方式保存界面文字和 prompt 内容，并通过 `QSettings` 记录当前语言。Render 则负责 HTML 转义、Markdown 转 HTML、JSON 提取、diff 可读化和反馈详情的渲染。

换句话说，显示给用户的文字内容是通过该模块管理的，方便开发也美化 ui 界面。

3 小组成员分工情况

成员	负责模块	主要工作
杨若辰	界面框架搭建	设计 Qt 主窗口、版本树、AI 分析页、反馈记录页，将项目的雏形框架搭建起来，给后续具体细节开发打好基础。
宋睿宇	AI 与 Prompt 的处理	封装 DeepSeek API 请求，设计反馈、反馈复查和启发式问题的 prompt，并解析大模型的回复内容。
全体成员	ui 界面优化	调整界面 ui，包括滚轮状态记忆，状态染色，功能分区。
宋睿宇	项目收尾与撰写报告	修复小 bug，整理 README 与本报告。
杨若辰	项目验收	检查 GitHub 仓库状态，确认代码提交完整、所有内容正常渲染、功能完整，模拟验收。

4 项目总结与反思

4.1 设计收获

本项目最大的收获是把大模型的能力加入一个相对完整的项目流程中。直接调用大模型并不难，难点在于如何组织上下文、如何让返回结果可解析、如何把反馈和代码版本关联起来。

另一个收获是 Git 底层运行逻辑。为了将版本控制加入我们的项目，我们首先花了两周时间研究 Git 底层的运行逻辑，包括一个 commit 包含的内容、Tree 节点的版本保存思路等，同时我们也学会了在日常学习中使用 Git、GitHub 管理我们的代码。

4.2 不足与改进方向

项目仍有一些可以继续完善的地方。比如当学生完成了整个学期的学习之后，项目可以开发一个功能，通过当前作业文件夹中所有版本的反馈记录生成一个复习建议以及学习报告，帮助学生从整体上掌握一整门课程的内容，不至于使得各次作业之间是独立的。

另外，还可以加入离线演示模式、测试用例自动生成等功能，使它更适合学生长期使用。